1. Методика построения операционных графовых моделей программ (ОГМП)

1.1. Основные этапы и способы построения ОГМП.

При построении ОГМП рассматривается представление программ в виде графов с нагруженными вершинами (ГНВ) или графов с нагруженными дугами (ГНД). В качестве нагрузки элементов графа будут выступать параметры L_i или L_{ij} (количество ресурса, потребляемого процессом, сопоставляемым i-ой вершине или ij-ой дуге) и вероятности P_{ij} выполнения программы по дуге ij.

Можно выделить три этапа, необходимых для построения ОГМП:

1) построение управляющего графа программы (УГП), который станет каркасом модели, требующим нагружения своих элементов потребляемыми ресурсами;
2) оценка параметров потребления ресурсов фрагментами программы, сопоставляемым вершинам или дугам графа;
3) оценка и задание вероятностей выбора маршрутов выполнения программы.

Построение УГП *(1 этап)* производится путем выделения в программе функциональных участков (ФУ) и сопоставления им элементов управляющего графа. Выполнение этого этапа вручную для небольших учебных программ не вызывает затруднений.

Возможна автоматизация построения УГП по тексту программы (на рынке ПО есть продукты, позволяющие выполнить построение моделей программ, написанных на процедурных языках, в виде структурного графа), но необходимо еще и нагрузить элементы полученного УГП, что является более сложной задачей построения модели и в известных продуктах отсутствует.

Оценка времен выполнения участков *(2 этап)* может производиться одним из следующих способов:

- прямым подсчетом по тексту программы в ассемблерном представлении с использованием времен выполнения команд, задаваемых в тактах процессора.
- с использованием программного измерительного монитора (ПИМ);

При оценке потребления ресурсов посредством прямого подсчета по ассемблерному коду программы возникают трудности из-за оптимизации программы современными компиляторами, затрудняющей сопоставление оператора ЯВУ и ассемблерного кода, в который он транслируется. При отключении оптимизации такое сопоставление провести можно, однако получаемые в этом случае оценки нельзя будет считать корректными.

При получении оценок потребления ресурса времени участком кода с использованием ПИМ таких проблем не возникает. Однако, при работе с профилировщиком в результаты измерений вносятся искажения из-за потребления им ресурсов ЭВМ на собственные нужды. При использовании ПИМ Sampler влияние этого негативного фактора может быть снижено, так как в данном мониторе реализована коррекция результатов измерений. Поэтому оценка потребления ресурса с помощью ПИМ является предпочтительной.

После выбора способа оценивания времени выполнения одного участка программы. необходимо организовать весь процесс нагружения модели параметрами потребления ресурсов, то есть:

- выбрать, какие фрагменты программы оценивать;
- оценить параметры потребления ресурсов заданными фрагментами;
- связать полученные ресурсные параметры с управляющим графом программы.

Третий этап построения модели программы, связанный с оценкой вероятностей для каждого ветвления в УГП, является наиболее трудным. В ряде случаев, выбор ветвей сложным образом зависит от структуры и параметров программы и не может быть оценен аналитически. Тогда применяется имитационное моделирование – значения параметров, влияющих на события, определяющие выбор ветвей, задаются с помощью датчика случайных чисел с требуемым законом распределения и методом статистических испытаний, определяются вероятности наступления событий.

Рассмотрим пример проведения испытаний и обработки их результатов. Допустим, в качестве модели фрагмента программы используется граф с нагруженными вершинами, соответствующий циклу WHILE, и требуется получить оценку вероятностей выбора ветвей входа в цикл P и выхода из цикла Q (рис. 1).

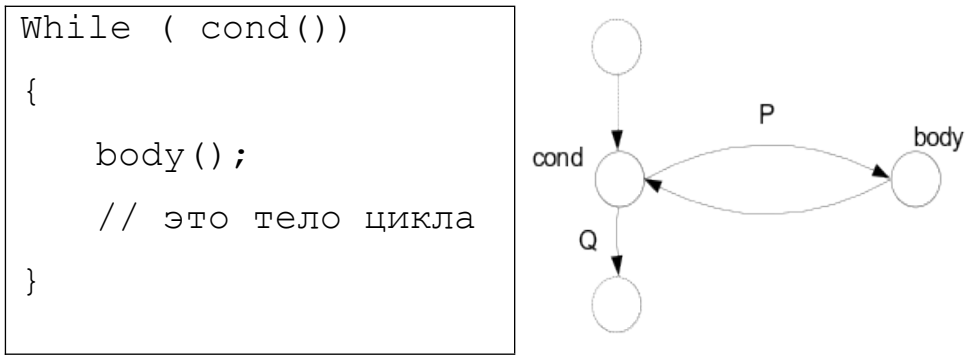


Рис. 1. Исследуемый фрагмент и его модель

С помощью профилировщика можно измерить параметры:

1) число выполнений данного фрагмента программы (параметры типа N_i);

2) количество передач управления данному участку и от него (параметры типа N_{ij}).

Пусть в исходный текст для контроля прохождения путей внедряются вызовы гипотетической функции CtrPoint(int Num), реализующей счетчик числа проходов (т.е. фиксирующей, сколько раз произошел вызов функции с конкретным номером Num), и пусть эти счетчики расставлены в тексте программы следующим образом

```
CtrPoint(1);
while ( cond())
{
   CtrPoint(2);
   body();
   // это тело цикла
   CtrPoint(3);
}
CtrPoint(4);
```

Тогда при прогоне программы на разных наборах входных данных могут быть получены, например, такие данные:

№ CtrPoint	Среднее число выполнений
1	5
2	30
3	30
4	5

Из этого можно вывести, что при проверке условия выполнения цикла значения true и false получены соответственно 30 и 5 раз ($N_{true}=N_2=N_3=30$, $N_{false}=N_1=N_4=5$), а в совокупности условие проверялось $N = N_{true}+N_{false} = 35$ раз. Тогда вероятности входа в цикл и выхода из него соответственно равны:

$$P = N_{true} / N = 30/35 = 6/7$$

$$Q = N_{false} / N = 5/35 = 1/7$$

Таким образом, чтобы получить вероятности ветвлений, необходимо знать значения “параметров типа N_i”. Проверим, достаточно ли их, чтобы оценить и потребление ресурсов в вершинах графа, или нужны также и “параметры типа N_{ij}”?

Для рассмотренной в примере конструкции вычисление условия выполнения цикла происходит в следующих случаях:

1) вход в цикл – первая проверка, результат true;

2) переход на новую итерацию – результат true;

3) выход из цикла – результат false;

4) обход мимо цикла – результат false.

Получается, что проверка условия, в ГНВ присутствующая в виде одной вершины, может осуществляться при одной из четырех передач управления, причем времена выполнения переходов могут быть различными. Тогда потребление ресурсов в вершине проверки условия определяется взвешенной суммой времен, соответствующих ей переходов:

$$\frac{\sum_{i,j} N_{ij} L_{ij}}{\sum_{i,j} N_{ij}},$$

где N_{ij} – количество переходов ij, относящихся к проверке условия;

L_{ij} – время выполнения одного перехода ij.

Допустим, есть два варианта выполнения программы при одном и том же распределении параметров:

i (номер перехода)	начало и конец перехода	L_i	N_i (1 вариант)	N_i (2 вариант)
1	1,2	3	5	3
2	3,2	5	25	27
3	3,4	3	5	3
4	1,4	5	0	2
5	*2,3*	-	*30*	*30*

Здесь нам важны переходы №№ 1–4, переход №5 соответствует выполнению тела цикла, и не учитывается при расчете параметров вершины принятия решения.

Расчет показывает, что для этих двух вариантов потребление ресурсов в вершине составит соответственно:

L(1) = (5*3+25*5+5*3+0*5) / (5+25+5+0) = 155 / 35 = 4.43 ед.

L(2) = (3*3+27*5+3*3+2*5) / (3+27+3+2) = 163 / 35 = 4.66 ед.

Через “параметры типа N_{ij}” также можно выразить и вероятности ветвлений:

$N_{true} = N_{1,2} + N_{3,2} = 30$

$N_{false} = N_{1,4} + N_{3,4} = 5$

$P = N_{true} / (N_{true} + N_{false}) = 6/7$

$Q = N_{false} / (N_{true} + N_{false}) = 1/7$

Итак, для оценки вероятностей ветвлений посредством имитационного моделирования необходимо фиксировать число передач управления между

функциональными участками программы. При этом использованная в примере функция CtrPoint(Num) должна запоминать значение параметра Num при ее предыдущем вызове, интерпретируя два соседних по времени вызова как переход между ФУ. Аналогичный прием используется в ПИМ Sampler.

1.2. Методика использования ПИМ Sampler для построения модели

В предлагаемой методике для выполнения этапов 2 и 3 построения модели (оценка вероятностей ветвлений и оценка времен выполнения фрагментов) предлагается использовать ПИМ Sampler, а для описания исследуемых программ следует выбрать язык C. Рассмотрим подробнее использование монитора Sampler на этих этапах.

В результате выполнения программы под управлением ПИМ Sampler данные об измерениях сохраняются в виде наборов записей [«номер строки измерения (i-1)», «номер строки измерения (i)», «общее время перехода между строками» (точнее процессов, выполнение которых включается в переход между строками), «количество переходов»].

В качестве функции CtrPoint(int) задания контрольной точки (КТ) на C/C++ реализована функция SAMPLE(void), которая определена в подключаемом заголовочном файле sampler.h как макрос:

```
/* sampler.h */
#ifndef NSAMPLER
  // ......
  #define SAMPLE    __throw_sampler(__FILE__,__LINE__)
  // ......
#endif   // NSAMPLER
```

То есть для выполнения измерения вызывается функция __throw_sampler, которой передаются имя исходного программного модуля и текущий номер строки (стандартные макросы __FILE__ и __LINE__). В качестве идентификатора КТ выступает номер строки размещения вызова соответствующей функции КТ, и пользователь ПИМ не имеет возможности при вызове задать другой номер.

Однако часто при расстановке КТ в тексте программы и последующем анализе результатов измерений удобнее оперировать не номерами строк, а номерами самих КТ. Для этого следует дополнить ПИМ Sampler требуемой функциональностью, определив свою функцию КТ и поместив ее описание в отдельный заголовочный файл:

```
/* ctrpoint.h */
#ifndef NSAMPLER
```

```
    #include "sampler.h"
  #ifndef CTRPOINT
    #define CTRPOINT (_sid_)  __throw_sampler(__FILE__,
(_sid_))
  #endif //CTRPOINT
  #endif   // NSAMPLER
```

Далее в качестве функции КТ будем применять функцию CTRPOINT(int).

Теперь записи об измерениях примут вид [«КТ i-1», «КТ i», «общее время перехода между строками», «количество переходов»]. Результатам измерений можно сопоставить граф с нагруженными дугами (будем называть его *графом переходов*), подобный ОГМП. В этом графе вершинам соответствуют вызовы функций КТ(i), дугам – переходы от одной функции измерения КТ(i-1) к другой КТ(i). При этом дуги нагружаются параметрами «количество переходов между КТ» и «время одного перехода между функциями КТ». Таким образом, граф переходов будет включать и параметры потребления ресурсов, и вероятности выбора маршрутов (которые могут быть вычислены, исходя из соотношения числа переходов между КТ).

Для того, чтобы граф переходов обеспечивал построение корректной ОГМП, необходима правильная расстановка КТ в тексте программы, учитывающая возможные маршруты ее выполнения. Контрольные точки должны быть установлены на границах линейных участков программы, а также - до и после границ блоков и управляющих операторов (if, while, do).

Таким образом, построение модели программы с использованием ПИМ Sampler требует выполнения трех стадий:

1. Построение УГП;
2. Расстановка КТ в исходном тексте;
3. Вычисление нагрузочных параметров элементов УГП на основе графа переходов.

В качестве представления УГП лучше использовать ГНД, так как это общая форма, из которой можно получить ГНВ и модель в виде стохастической сети Петри (ССП).

На основе графа переходов (отчета с результатами измерения ПИМ Sampler в заданных КТ) и описания УГП, формируется описание ОГМП заданного типа, являющейся исходной для последующего расчета характеристик производительности программы. Частичная автоматизация построения ОГМП описана в разделе 2.

1.3. Связь графа переходов с ОГМП для типовых конструкций

Рассмотрим, как связаны между собой граф переходов, ГНД и ГНВ, и правила расчета нагрузочных параметров (вероятности ветвлений и потребления ресурсов) элементов графов для типовых управляющих конструкций языка С:

- линейная последовательность операторов (“линейный участок”)
- ветвление (if);
- ветвление с альтернативой (if-else);
- цикл с предусловием (while);
- цикл с постусловием (do-while);
- цикл со счетчиком (for).

Этих конструкций достаточно для построения структурных программ, а нарушающие принципы структурного программирования конструкции передачи управления (типа goto, break, continue, return) здесь рассматриваться не будут.

Конструкция Линейный участок

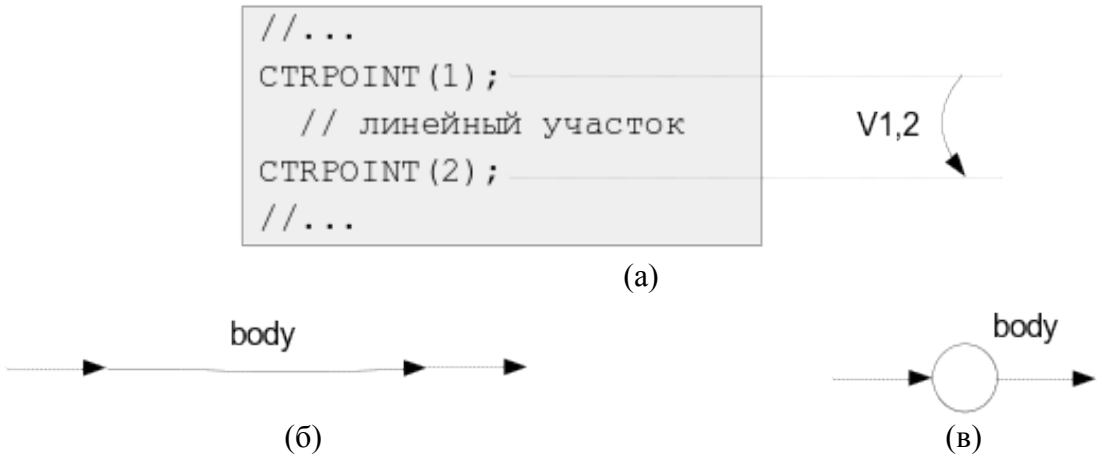


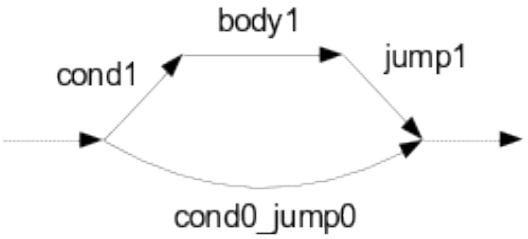
Рис. 5. Граф переходов (а), ГНД (б) и ГНВ (в) для линейного участка

Очевидно, что $L_{ГНВ}(body) = L_{ГНД}(body) = L_{1,2}$.

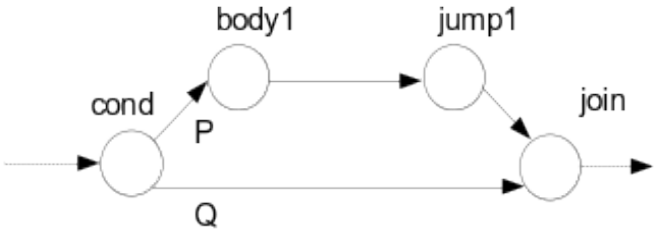
Конструкция «ветвление» (if)

```
//...
CTRPOINT(1);
  if ( cond() ) {
CTRPOINT(2);
    // тело "then"
CTRPOINT(3);
  }
CTRPOINT(4);
//...
```

(а)



(б)



(в)

Рис. 6. Граф переходов (а), ГНД (б) и ГНВ (в) для ветвления

Пересчет параметров при переходе от графа переходов к ГНД:

$P(cond1) = N_{1,2} / (N_{1,2} + N_{1,4})$

$P(cond0_jump0) = 1 – P(cond1) = N_{1,4} / (N_{1,2} + N_{1,4})$

$L(cond1) = L_{1,2}$ $L(body1) = L_{2,3}$

$L(jump1) = L_{3,4}$ $L(cond0_jump0) = L_{1,4}$

Пересчет параметров при переходе от ГНД к ГНВ:

$P = P(cond1)$

$Q = 1 – P$

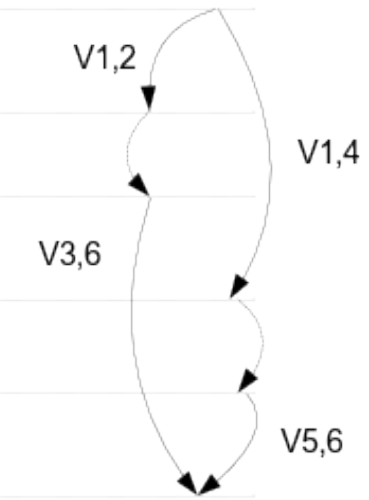
$L(cond) = P*L(cond1) + Q*P(cond0_jump0)$

$L(jump1) = L_{ГНД}(jump1) = L_{3,4}$

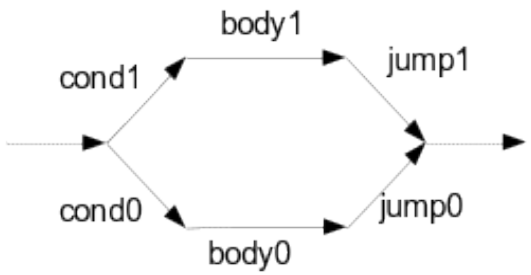
$L(body1) = L_{2,3}$ $L(join) = 0$

Конструкция Ветвление с альтернативой (if-else)

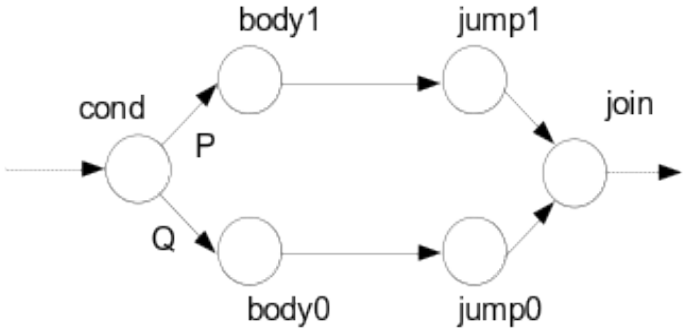
```
//...
CTRPOINT(1);
  if ( cond() ) {
CTRPOINT2);
    // тело "then"
CTRPOINT(3);
  } else {
CTRPOINT(4);
    // тело "else"
CTRPOINT(5);
  }
CTRPOINT(6);
//...
```



(а)



(б)



(в)

Рис. 7. Граф переходов (а), ГНД (б) и ГНВ (в) для ветвления с альтернативой

Пересчет параметров при переходе от графа переходов к ГНД:

$P(cond1) = N_{1,2} / (N_{1,2} + N_{1,4})$

$P(cond0) = 1 – P(cond1) = N_{1,4} / (N_{1,2} + N_{1,4})$

$L(cond1) = L_{1,2}$ $L(body1) = L_{2,3}$

$L(cond0) = L_{1,4}$ $L(body2) = L_{4,5}$

$L(jump1) = L_{3,6}$

$L(jump0) = L_{5,6}$

Пересчет параметров при переходе от ГНД к ГНВ:

$P = P(cond1)$

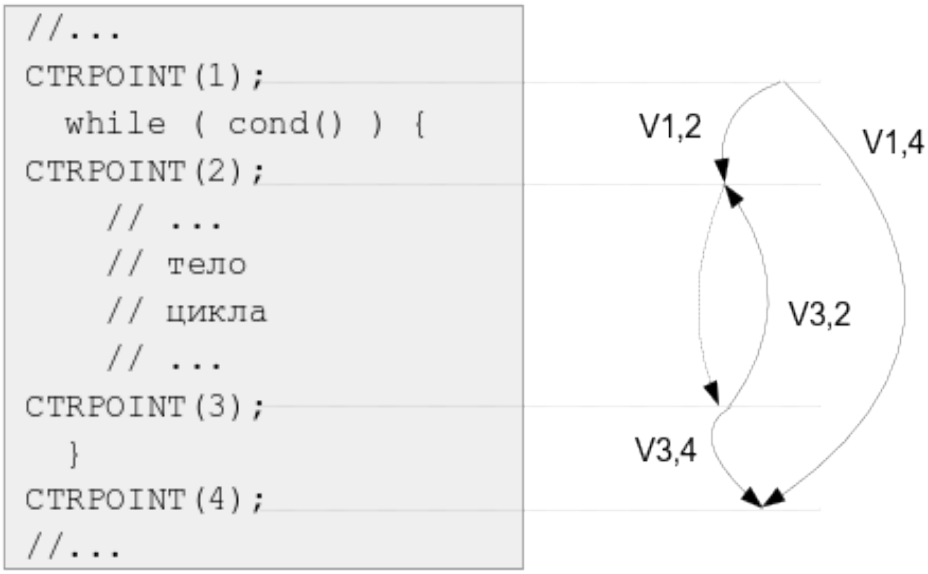
$Q = 1 – P$

$L(cond) = P^* L(cond1) + Q^* L(cond0)$

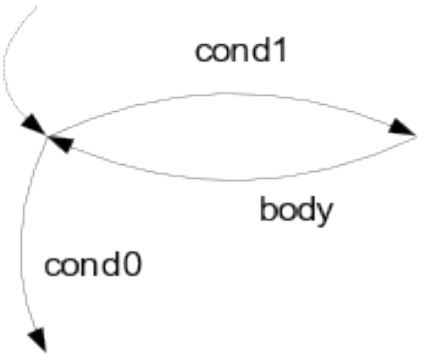
$L(body1) = L_{2,3}$ $L(jump1) = L_{ГНД}(jump1)$

$L(body2) = L_{4,5}$ $L(jump0) = L_{ГНД}(jump0)$

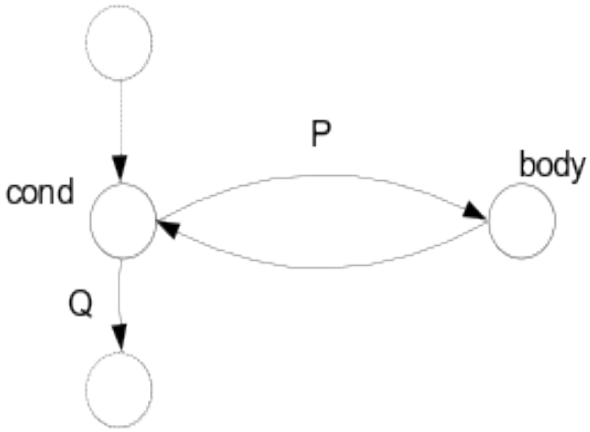
Конструкция Цикл с предусловием (while)



(а)



(б)



(в)

Рис. 8. Граф переходов (а), ГНД (б) и ГНВ (в) для цикла с предусловием

Пересчет параметров при переходе от графа переходов к ГНД:

$N_{cond} = N_{1,2} + N_{1,4} + N_{3,2} + N_{3,4}$

$P(cond1) = (N_{1,2} + N_{3,2}) / N_{cond}$

$P(cond0) = 1 – P(cond1) = (N_{1,4} + N_{3,4}) / N_{cond}$

$L(cond1) = (N_{1,2}*L_{1,2} + N_{3,2}*L_{3,2}) / (N_{1,2} + N_{3,2})$

$L(cond0) = (N_{1,4}*L_{1,4} + N_{3,4}*L_{3,4}) / (N_{1,4} + N_{3,4})$

Пересчет параметров при переходе от ГНД к ГНВ:

$P = P(cond1)$

$Q = 1 – P$

$L(cond) = P* L(cond1) + Q* L(cond0)$

Конструкция Цикл с постусловием (do-while)

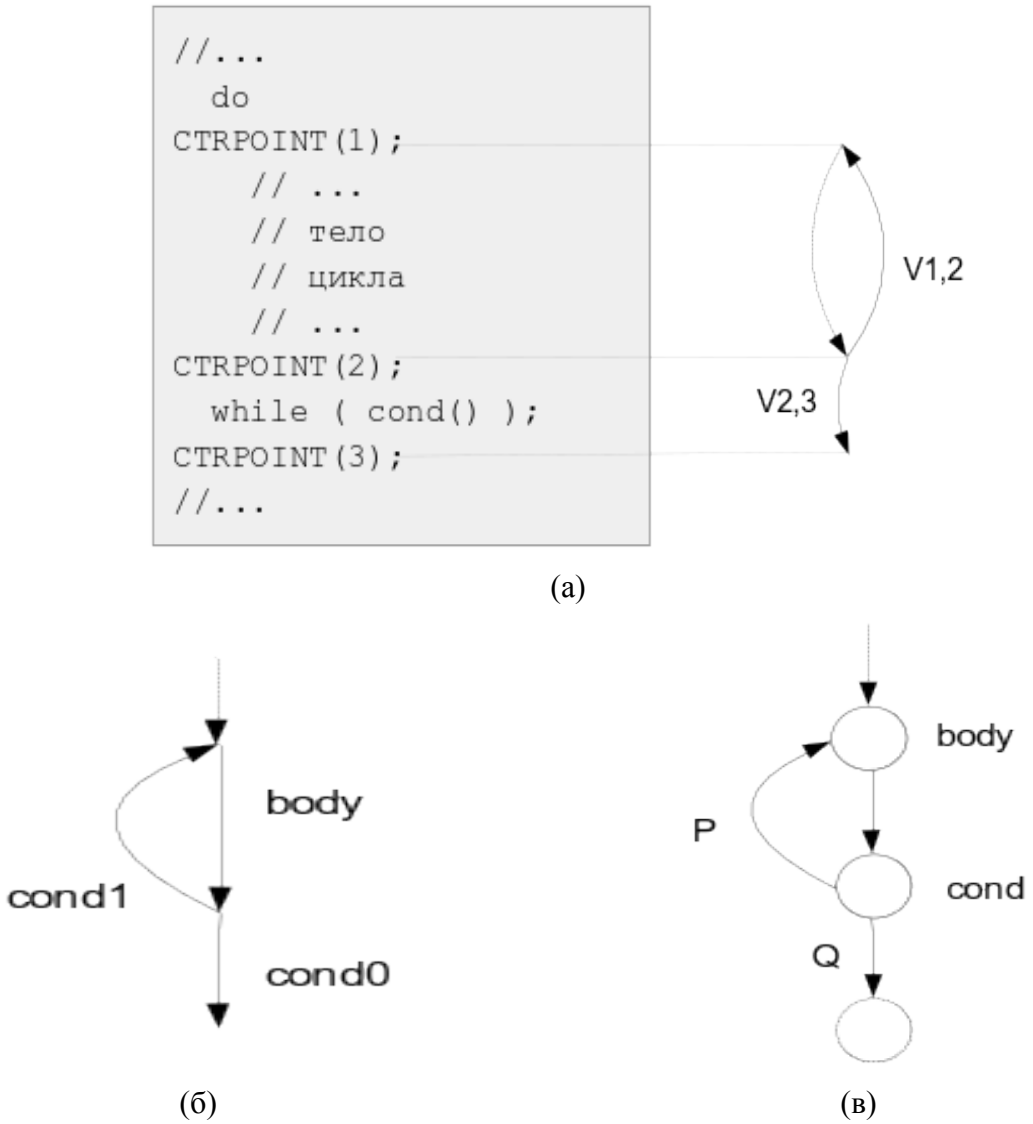


Рис. 9. Граф переходов (а), ГНД (б) и ГНВ (в) для цикла с постусловием

Пересчет параметров при переходе от графа переходов к ГНД:

$P(cond1) = N_{2,1} / (N_{2,1} + N_{2,3})$

$P(cond0) = 1 – P(cond1) = N_{2,3} / (N_{2,1} + N_{2,3})$

$L(cond1) = L_{2,1}$

$L(cond0) = L_{2,3}$

Пересчет параметров при переходе от ГНД к ГНВ:

$P = P(cond1)$

$Q = 1 – P$

$L(cond) = P^* L(cond1) + Q^* L(cond0)$

Конструкция Цикл со счетчиком (for)

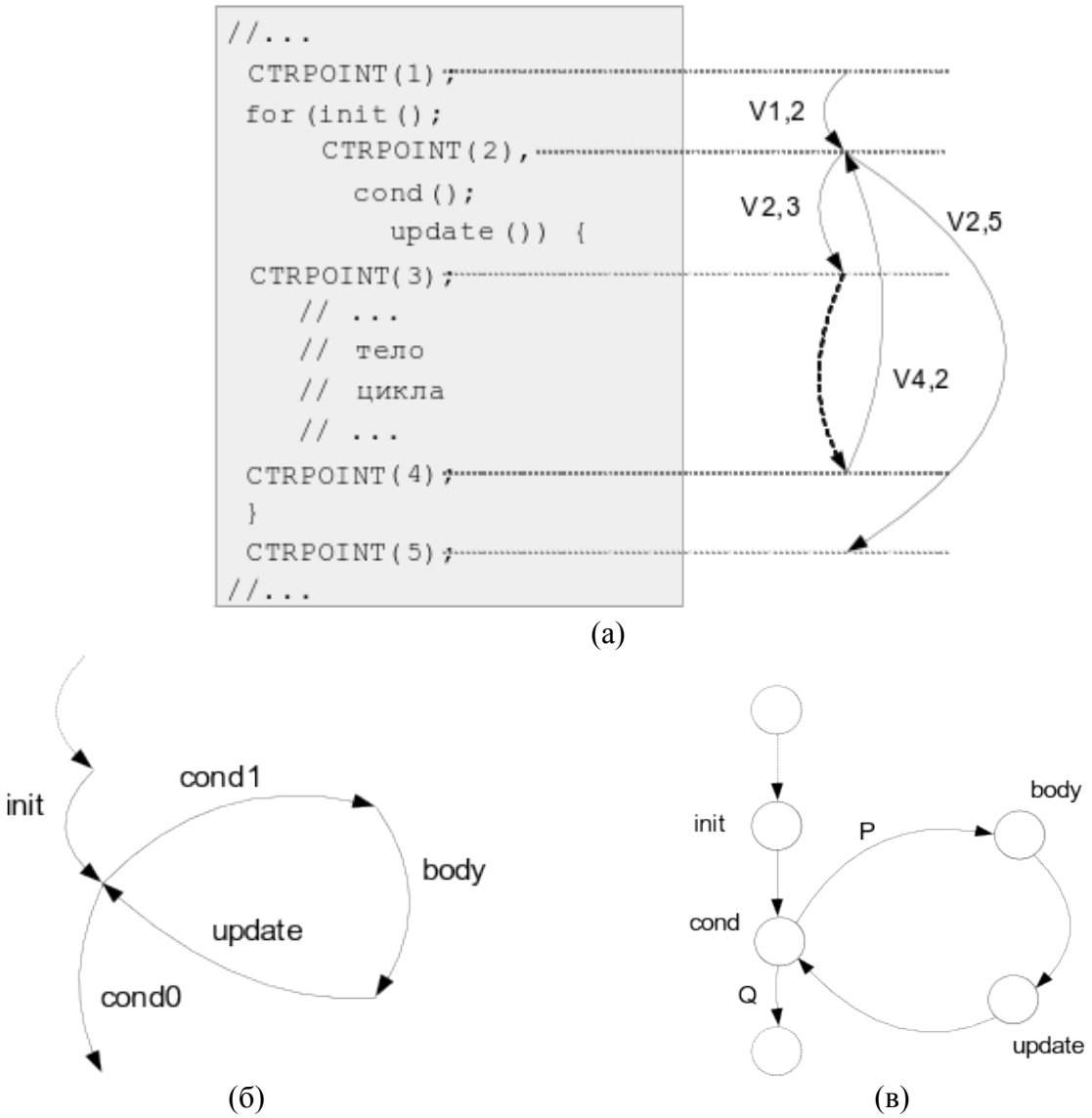


Рис. 10. Граф переходов (а), ГНД (б) и ГНВ (в) для цикла со счетчиком

Пересчет параметров при переходе от графа переходов к ГНД:

$P(cond1) = N_{2,3} / (N_{2,3} + N_{2,5})$

$P(cond0) = 1 – P(cond1) = N_{2,5} / (N_{2,3} + N_{2,5})$

$L(cond1) = L_{2,3}$

$L(cond0) = L_{2,5}$

$L(init) = L_{1,2}$

$L(update) = L_{4,2}$

Пересчет параметров при переходе от ГНД к ГНВ:

$P = P(cond1)$

$Q = 1 – P$

$L(cond) = P^* L(cond1) + Q^* L(cond0)$

$L(init) = L_{ГНД}(init)$

$L(update) = L_{ГНД}(update)$

2. Реализация программной поддержки построения ОГМП

2.1. Общие принципы реализации

Средство поддержки построения ОГМП, выполнено в виде программного модуля, встроенного в пакет анализа производительности CSA3. Этот модуль, как и пакет CSA3, написан на объектно-ориентированном языке Ruby и взаимодействует с пользователем посредством графического интерфейса. В этом разделе под целевой программой понимается функция на языке C/C++ (main или любая другая), подвергаемая анализу.

Основной частью реализации является блок “Расстановка КТ” (рис. 11). Этот блок принимает на входе исходный текст целевой программы и на выходе формирует модифицированный текст программы (с внедренными вызовами функции КТ) и управляющий граф программы, нагруженный данными о переходах между КТ, по которым позднее будет производиться вычисление параметров ОГМП.

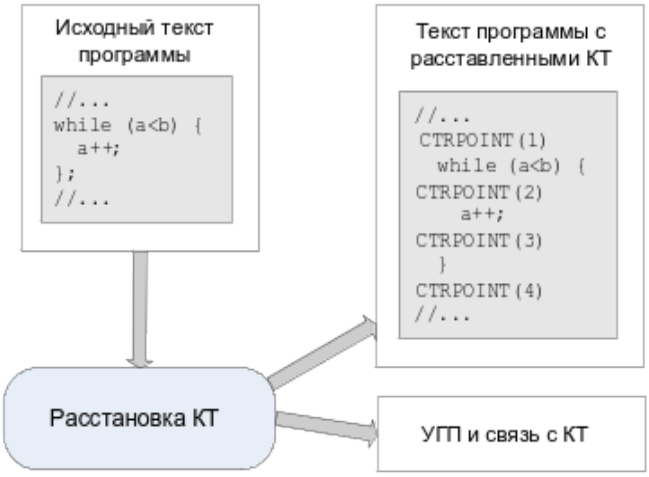


Рис. 21. Схема работы блока“ Расстановка КТ”

По сути, данный блок является транслятором исходного текста программы в другие формы ее представления. В процессе перевода осуществляется построение и анализ целевой программы на нескольких уровнях представления:

1) уровень непосредственно исходного текста;
2) уровень токенов (токен определяет тип лексемы и ее позицию в исходном тексте);
3) уровень синтаксического дерева;
4) уровень модели (УГП).

При переходе от первого уровня ко второму выполняется лексический анализ, на выходе которого формируется поток токенов. При переходе от второго уровня к третьему осуществляется синтаксический анализ потока токенов и формируется синтаксическое

дерево программы. Это дерево отражает структуру программы как совокупности вложенных друг в друга конструкций языка. В дереве каждая конструкция (узел дерева) “знает” о своем местоположении в исходном коде (которое задается токенами, ограничивающими эту конструкцию и ее тело).

При переходе на уровень модели в ходе анализа структуры дерева происходит:

- внедрение в исходный текст вызовов функции КТ, необходимых для расчета параметров подграфа текущей конструкции. При этом в случае необходимости (рис. Error: Reference source not found2) вставляются недостающие операторные скобки;

```
if (cond())
  simple_body();
```

➔

```
CTRPOINT(1);
if (cond()) {
  CTRPOINT(2);
  simple_body();
  CTRPOINT(3);
}
CTRPOINT(4);
```

Рис. 12. Участок кода, требующий добавления операторных скобок

- построение УГП, дуги которого для каждой конструкции нагружаются данными о переходах между КТ.

Опишем *принцип задания связи УГП и графа переходов.*

Обозначим как T[i, j] переход между КТ с номерами i и j. Параметры дуги линейного участка (вершина-источник которой имеет только одну исходящую дугу) определяются одним переходом:

$$L = L(T[i, j]) = L_{ij}\ ; \quad P = 1.$$

Параметры дуги принятия решения зависят от нескольких переходов, которые могут принадлежать одному из множеств:

- *основному множеству UP*, включающему переходы, соответствующие выбору данной дуги.
- *дополняющему множеству DOWN*, включающему переходы, соответствующие выбору дуги, альтернативной по отношению к данной дуге.

Через параметры переходов основного множества вычисляется потребление ресурсов данной дугой:

$$L = (\sum N_{ij} L_{ij}) / (\sum N_{ij}), \text{ где } T[i, j] \in UP ;$$

Параметры переходов из дополнительного множества (совместно с переходами основного множества) необходимы для вычисления вероятности:

$$P = (\sum N_{ij}) / (\sum N_{ij} + \sum N_{mn}), \text{ где } T[i, j] \in UP, T[m, n] \in DOWN.$$

Заметим, что для дуги линейного участка:

$$UP = T[i, j] \; ; \qquad DOWN = \varnothing .$$

Таким образом, для задания нагрузки дуг УГП достаточно указать переходы, входящие во множества UP и DOWN данной дуги.

После расстановки КТ модифицированный текст исследуемого фрагмента вставляется в исходный текст программы. В результате исполнения скомпилированной программы под управлением монитора Sampler формируется отчет, содержащий данные о выполненных переходах между КТ (граф переходов).

Блок “Нагружение УГП” (рис. 13) на входе принимает созданный Sampler’ом граф переходов и построенное ранее описание УГП, затем, используя соотношения из 1.3, формирует описание модели программы в виде ГНД.

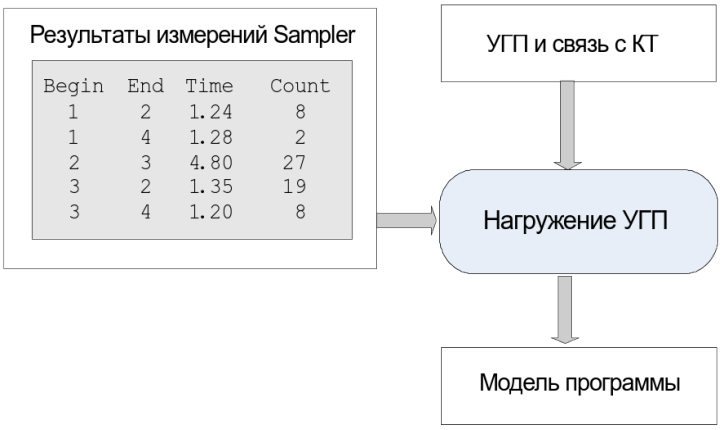


Рис. 3. Схема работы блока “ Нагружение УГП”

Полученная модель программы далее может быть преобразована в ГНВ, ССП или непосредственно использована в качестве исходной для выбранного метода анализа производительности из методов, реализованных в пакете CSA3.

2.2. Использование средства поддержки построения ОГМП

В реализованном средстве поддержки построения ОГМП создание управляющего графа и разметка текста программы осуществляются поэтапно. Сначала программа

представляется в виде единственной дуги, сопоставляемой всему телу программы. Далее дуга, если ей соответствует последовательность конструкций, заменяется графом из последовательно соединенных дуг этих конструкций. Если дуге соответствует конструкция с “вложением” (цикл или ветвление), то дуга заменяется подграфом этой конструкции. Процесс замены дуг на подграфы конструкций должен управляться пользователем и завершается при достижении требуемого уровня детализации структуры программы (не обязательно “раскрывать” все конструкции).

Запуск средства поддержки построения ОГМП производится из главного окна CSA3 (рис. 14) посредством кнопки , при нажатии на которую открывается окно (рис. 55), функциональность которого реализует этап “Расстановка КТ”.

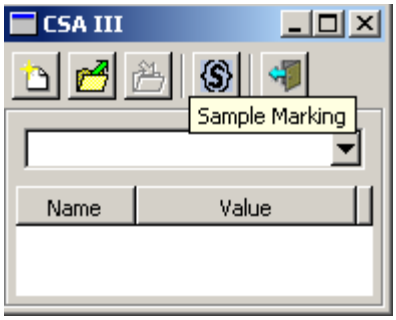


Рис. 44. Главное окно дополненной версии системы CSA3

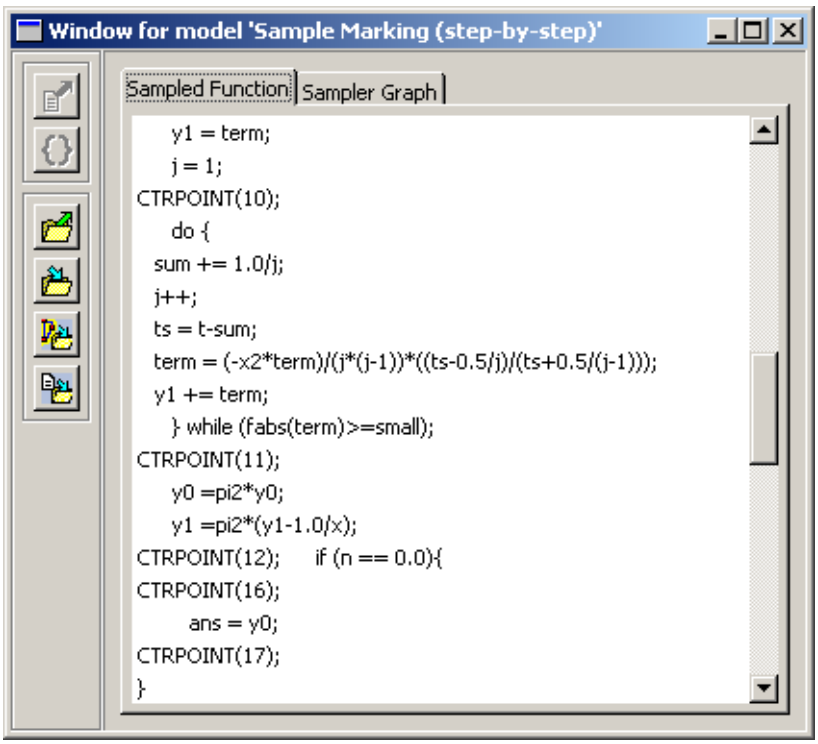
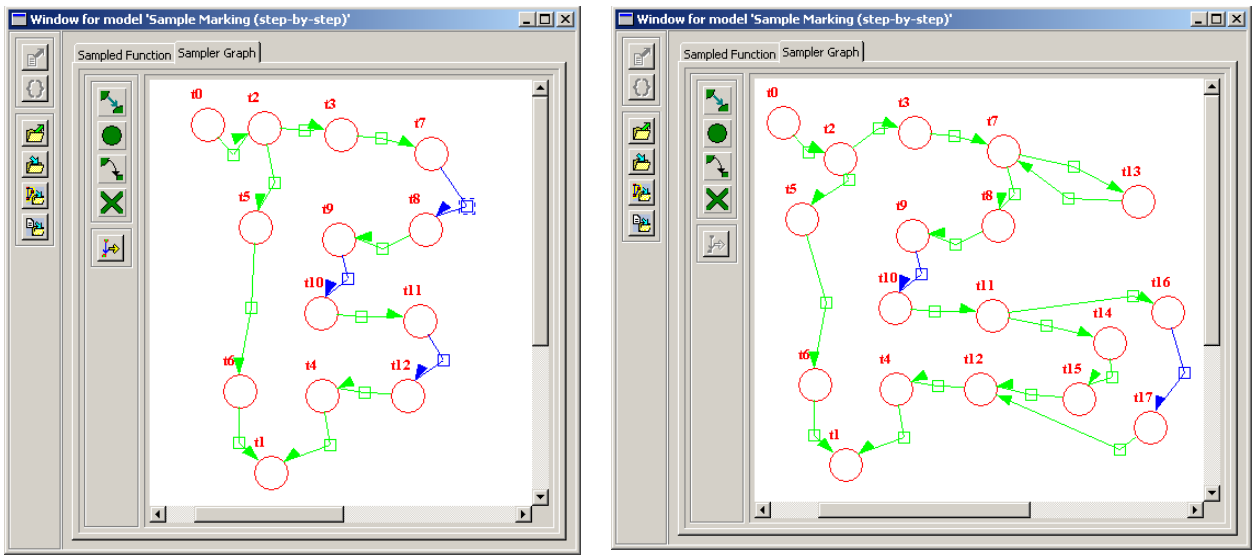


Рис. 55. Окно блока “Расстановка КТ”

Окно разделено на две зоны, в которых расположены рабочая область и кнопочная панель. В рабочей области могут находиться вкладки, содержащие:

1) весь исходный текст программы, загружаемый в начале работы;
2) текущий текст исследуемой функции с внедренными КТ, изменяемый в процессе детализации модели;
3) графический редактор – включает в себя область рисования графа текущей исследуемой функции и кнопочную панель, содержащую стандартные кнопки редактирования графа и кнопку “раскрытия дуги” . Эта кнопка заменяет “нераскрытую” дугу (синего цвета) на соответствующий подграф (рис.66).



(а) (б)

Рис. 66. Граф до (а) и после (б) “раскрытия дуг”

Кнопочная панель содержит следующие кнопки:

– загрузка из файла исходного текста программы;

– выбор исследуемой функции, построение начального графа (состоящего из одной дуги) и начальной разметки КТ (два вызова КТ по границам функции);

– загрузка модели (УГП + текст функции) из файла с описанием (*.sgm);

– сохранение описания модели в файл (*.sgm);

– сохранение описания УГП в файл (*.xml, *.csa3);

– сохранение модифицированного текста функции в файл.

При выделении дуги в инспекторе объектов главного окна (рис. 77) отображаются данные об этой дуге:

- name – имя дуги;
- info – какой части конструкции соответствует дуга;

- up, down – переходы, относящиеся ко множествам UP и DOWN, необходимые для расчета параметров дуги в ГНД.

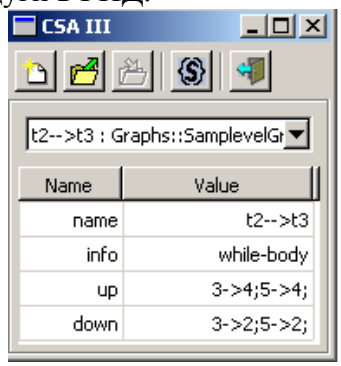


Рис. 77. Данные о дуге в инспекторе объектов

Данные “up” и “down” в процессе детализации модели автоматически формируются только для “дуг линейных участков”, а данные для “дуг принятия решения” должны вводиться вручную. Это позволяет обучаемым применять свои знания по проблемам построения и анализа ОГМП. Данные отображаются в инспекторе объектов главного окна при выделении дуги, в полях ввода “up” и “down”:

- в поле “up” заносятся переходы, соответствующие выбору данной дуги;
- в поле “down” заносятся переходы, соответствующие выбору дуги, альтернативной по отношению к данной дуге.

Наименования переходов в поля ввода вносятся в формате “src->dst;..” , например “1->4;3->4;”.

При загрузке из файла сохраненного описания УГП (кнопка главного окна) открывается окно (рис. Error: Reference source not found8), реализующее функциональность этапа “Нагружение УГП”.

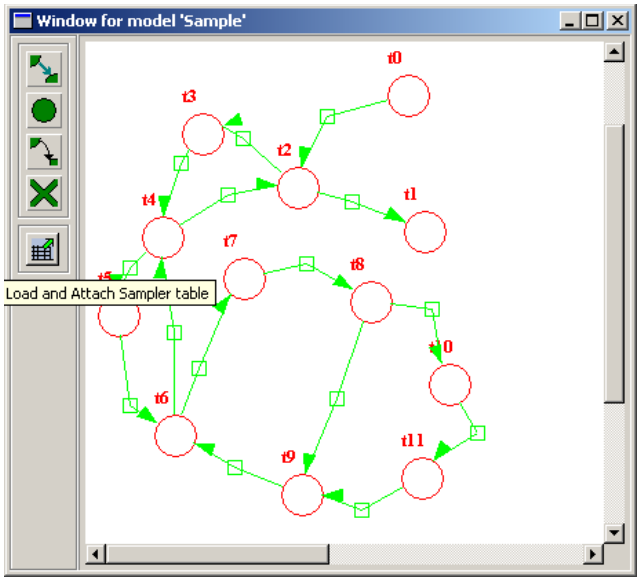


Рис. 18. Окно блока “Нагружение УГП”

Это окно также содержит область рисования графа и внутреннюю кнопочную панель со стандартными кнопками редактирования структуры графа. На кнопочной панели расположена кнопка , которая инициирует загрузку данных о переходах (файл отчет Sampler в формате SMP) и нагружение УГП вычисляемыми значениями вероятности ветвлений и потребления ресурсов. Получаемый в результате ГНД открывается в новом окне, в котором доступно применение к ГНД методов анализа производительности на базе аппарата Марковских цепей.

3. Проверка адекватности построения моделей

Суть проверки адекватности модели состоит в сравнении оценок количества потребляемого программой ресурса (времени выполнения), полученных:

1) в результате непосредственного измерения времени выполнения программы;
2) методами расчета оценок по построенной ОГМП.

Рис.19. Методика тестирования адекватности построения ОГМП

В первом случае на границах тела программы внедряются 2 вызова функции КТ и осуществляется измерительный эксперимент – выполнение программы на нескольких различных наборах входных данных под управлением монитора Sampler. В качестве результата измерения будет выступать $L_{изм}$ – среднее время выполнения программы.

Во втором случае применяется аналитический расчет времени выполнения по модели программы, сформированной с использованием средства поддержки построения ОГМП. Полученная оценка количества потребляемого ресурса $L_{теор}$ сравнивается со значением $L_{изм}$. Близкие значения $L_{теор}$ и $L_{изм}$ свидетельствуют об адекватном представлении моделью динамики выполнения программы, а значит и о корректности разработанной методики построения ОГМП в целом.

При анализе модели методом эквивалентных преобразований возможно использование еще одной проверки. В соответствии с этим методом в результате серии преобразований (склеивание параллельных дуг, удаление циклических дуг, удаление вершин) граф сворачивается до одной дуги, параметры которой характеризуют потребление ресурсов моделируемым процессом. Для этой дуги вероятность должна равняться единице, иное значение будет свидетельствовать о некорректном назначении вероятностей дугам исходной модели.